

Prompt:

1. Torneos. La organización realiza múltiples torneos. Cada torneo tiene un nombre, el título del video-juego que se juega, fechas de inicio y fin, y un pozo de premios (prize pool) en USD. Además tiene un número máximo de equipos por participar.
2. Jugadores. Los jugadores se registran en la plataforma con un gamertag (único), su nombre real, email, fecha de nacimiento y país de origen.
3. Equipos. Los jugadores forman equipos (de un número no acotado de jugadores). Cada equipo tiene un nombre único, una fecha de creación y un capitán. El capitán debe ser miembro del equipo. Un jugador pertenece a exactamente un equipo (la membresía es fija, no cambia entre torneos).
4. Inscripción a torneos. Los equipos se inscriben en torneos. Cada torneo tiene un número máximo de equipos que pueden participar. Un equipo puede inscribirse en múltiples torneos.
5. Partidas. Dentro de un torneo se juegan partidas (matches). Cada partida enfrenta a exactamente dos equipos distintos, tiene una fecha y hora programada, un resultado (puntaje equipo A y puntaje equipo B) y una fase del torneo. Las fases posibles son: fase de grupos, cuartos de final, semifinal, final. El formato del torneo es: 8 equipos divididos en 2 grupos de 4, round-robin dentro de cada grupo, los 2 mejores de cada grupo avanzan a semifinales, luego final.

6. Estadísticas individuales. Para cada partida se registran las estadísticas de cada jugador de los equipos participantes: KOs, restarts y assists.

7. Sponsors. Empresas pueden auspiciar torneos. De cada sponsor se conoce su nombre, industria (tecnología, bebidas, ropa, etc.) y el monto en USD que aporta a cada torneo que auspicia. Un sponsor puede auspiciar múltiples torneos.

A partir de esta información, crea un esquema relacional; debe incluir tablas, atributos, tipos de datos, claves primarias y foráneas, y restricciones relevantes.

Este es el esquema relacional detallado basado en las reglas de negocio proporcionadas. Se ha diseñado para garantizar la integridad referencial y cumplir con las restricciones específicas (como la membresía única de jugadores y las fases de los torneos).

Esquema Relacional: Gestión de Torneos de E-sports

1. Tabla **Sponsors**

Almacena la información general de las empresas auspiciadoras.

- **ID_Sponsor** (INT, PK, AUTO_INCREMENT)
- **Nombre** (VARCHAR(100), UNIQUE, NOT NULL)
- **Industria** (VARCHAR(50))

2. Tabla **Equipos**

Un equipo tiene un nombre único y un capitán. Nota: La FK del capitán se añade después o se permite nula inicialmente para evitar problemas de circularidad al insertar.

- **Nombre_Equipo** (VARCHAR(100), PK)
- **Fecha_Creacion** (DATE, NOT NULL)
- **ID_Capitan** (VARCHAR(50), FK, NOT NULL) — *Apunta a*

Jugadores(Gamertag)

3. Tabla **Jugadores**

Cada jugador pertenece a exactamente un equipo.

- **Gamertag** (VARCHAR(50), PK)
- **Nombre_Real** (VARCHAR(100), NOT NULL)
- **Email** (VARCHAR(100), UNIQUE, NOT NULL)
- **Fecha_Nacimiento** (DATE, NOT NULL)
- **Pais** (VARCHAR(50))
- **Nombre_Equipo** (VARCHAR(100), FK, NOT NULL) — *Referencia a Equipos*

4. Tabla **Torneos**

Define el marco del evento y el límite de equipos.

- **ID_Torneo** (INT, PK, AUTO_INCREMENT)
- **Nombre** (VARCHAR(100), NOT NULL)
- **Videojuego** (VARCHAR(100), NOT NULL)
- **Fecha_Inicio** (DATE, NOT NULL)
- **Fecha_Fin** (DATE, NOT NULL)
- **Prize_Pool** (DECIMAL(12, 2))
- **Max_Equipos** (INT, DEFAULT 8)
- **CONSTRAINT** **chk_fechas** CHECK (Fecha_Fin >= Fecha_Inicio)

5. Tabla **Sponsor_Torneo** (Relación N:M)

Registra qué patrocinador aporta a qué torneo y cuánto.

- **ID_Sponsor** (INT, PK, FK)

- **ID_Torneo** (INT, PK, FK)
- **Monto_Aporte** (DECIMAL(12, 2), NOT NULL)

6. Tabla **Inscripciones** (Relación N:M)

Relaciona los equipos con los torneos en los que participan.

/* Se puede determinar un torneo usando id_torneo y nombre_equipo */ comentario humano

- **ID_Torneo** (INT, PK, FK)
- **Nombre_Equipo** (VARCHAR(100), PK, FK)
- **Fecha_Inscripcion** (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

7. Tabla **Partidas** (Matches)

Registra los enfrentamientos. Las fases están restringidas por un **CHECK**.

- **ID_Partida** (INT, PK, AUTO_INCREMENT)
- **ID_Torneo** (INT, FK, NOT NULL)
- **Equipo_A** (VARCHAR(100), FK, NOT NULL)
- **Equipo_B** (VARCHAR(100), FK, NOT NULL)
- **Fecha_Hora** (DATETIME, NOT NULL)
- **Score_A** (INT, DEFAULT 0)
- **Score_B** (INT, DEFAULT 0)
- **Fase** (VARCHAR(20))

- **CONSTRAINT** `chk_fase` CHECK (Fase IN ('fase de grupos', 'cuartos de final', 'semifinal', 'final'))

- **CONSTRAINT** `chk_distintos` CHECK (Equipo_A <> Equipo_B)

8. Tabla `Estadisticas_Jugador`

Registra el rendimiento individual en cada partida específica.

- **ID_Partida** (INT, PK, FK)
 - **Gamertag** (VARCHAR(50), PK, FK)
 - **KOs** (INT, DEFAULT 0)
 - **Restarts** (INT, DEFAULT 0)
 - **Assists** (INT, DEFAULT 0)
-

Resumen de Restricciones Lógicas

1. **Membresía Fija:** La columna `Nombre_Equipo` en la tabla `Jugadores` es **NOT NULL**, lo que obliga a que todo jugador pertenezca a un equipo desde su registro y no pueda quedar "libre" en el esquema.

2. **Capitanía:** El **ID_Capitan** en la tabla **Equipos** es una clave foránea que referencia a **Jugadores(Gamertag)**. Para asegurar que el capitán pertenece al equipo, se debería implementar un *trigger* a nivel de base de datos, ya que el esquema relacional estándar no puede validar que el capitán esté en la lista de jugadores de ese mismo equipo sin lógica adicional.

3. **Formato del Torneo:** Aunque el esquema permite cualquier fase, la regla de negocio del torneo (8 equipos, 2 grupos) debe controlarse mediante la lógica de creación de partidas en la tabla **Partidas**.

Nuestro Análisis:

Consideramos que el LLM hizo un buen trabajo a la hora de diseñar el esquema general de las tablas y las relaciones necesarias para poder acceder a los datos desde múltiples queries. Sin embargo, tiene un pequeño error: a pesar de que el número por defecto de equipos sea ocho, no existe ningún tipo de restricción que no permita aumentar este número, lo cual podría crear algún tipo de inconsistencia. Además, aunque no genere problemas, creó “Fecha_Inscripcion (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)” perteneciente a la tabla Inscripciones que no es un atributo necesario para la tabla de inscripciones. Ahora bien, todo el resto de lo pedido está muy bien logrado, los atributos tienen rangos y valores predeterminados adecuadamente puestos, muestra todos los atributos pedidos en el prompt, las relaciones son claras y precisas.

Algo a destacar es la tabla de partidas, que incluye todos los datos pedidos, el puntaje de cada equipo separados como ints para facilitar los cálculos a posterioridad. Además, no incluyó una tabla adicional para hacer la relación con la tabla de “Equipos”. Incluso nosotros, al hacer nuestros diseños personales no tuvimos en cuenta que, a la hora de hacer una query que necesitará un join de equipo y partida, bastaba con hacer join con una condición disyuntiva, es decir, un OR entre la id del equipo en cuestión con “Equipo_A” o “Equipo_B”.

Con respecto a las restricciones de integridad nos resulta impresionante lo bien que diseñó esto, cumpliendo con los aspectos mencionados en la clase de Diseño de Bases de Datos. Si observamos las tablas Torneos y Partidas vemos que existen constraints. En el caso de la tabla de Torneos se tiene que la fecha de inicio no puede ser mayor a la fecha de fin. Para la tabla Partidas se puede ver que una partida debe pertenecer a alguna fase de las que fue mencionada en el prompt, y también que en una partida los dos equipos que se enfrentan deben ser distintos.

Es también importante mencionar como el LLM trabajó con las llaves y hace funcionar la Dependencia Funcional, vista en la clase 7. Observamos que las tablas tienen llaves primarias y extranjeras, con las cuales es posible conseguir datos de otras tablas. Por ejemplo, las partidas dependen de que existan equipos y un torneo, los cuales son asociados a la tabla con una llave foránea y tienen la restricción de NOT NULL, esto refleja lo ya mencionado y ayuda a conocer los datos sobre las otras tablas. Esto se repite con las tablas de Estadísticas Jugador, Inscripciones, etc.

Finalmente, con todo lo mencionado, podemos observar que la LLM hizo un gran trabajo creando el diseño para la base de datos usando lo que le fue indicado en el prompt. No solo eso, también tuvo buenas prácticas, teniendo en cuenta restricciones que no fueron mencionadas en los requisitos (como que un equipo no puede enfrentarse a sí mismo y que las partidas deben pertenecer a alguna fase del torneo). Se ha de indicar que solo cometió un error que vale la pena mencionar, el cual podría llevar a tener una cantidad enorme e incorrecta de equipos, además de crear un atributo innecesario (aunque no problemático). Concluimos que, con supervisión y seguimiento, es posible usar a la IA para generar un esquema para una base de datos.